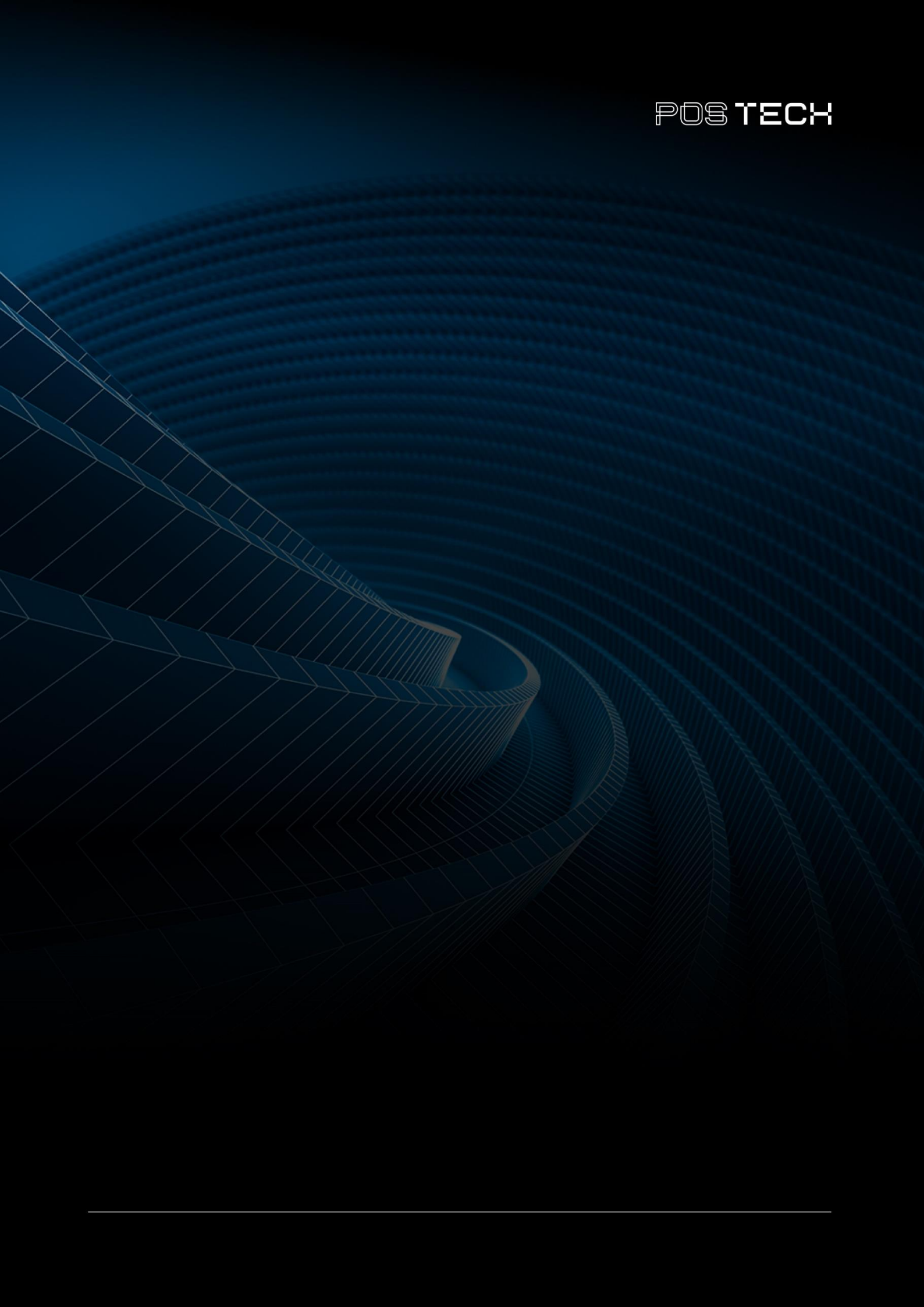

POS TECH

TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

O PROBLEMA

A FIAP Cloud Games (FCG) é agora uma plataforma funcional, distribuída e observável. No entanto, para atingir um nível profissional e suportar um crescimento global, a infraestrutura precisa superar três gargalos críticos:

1. **Ambiente de Produção Gerenciado:** garantir o uso do Kubernetes na Cloud em favor da resiliência e da segurança.
2. **Automação de Entrega (CI/CD):** eliminar processos manuais lentos e suscetíveis a erros.
3. **Performance e Latência:** com o aumento do tráfego, a dependência exclusiva de bancos relacionais e buscas simples tornou-se um impeditivo. É necessário otimizar a persistência de dados e a velocidade de resposta aos usuários.

O DESAFIO

Esta fase foca em automação, performance e portabilidade. O desafio consiste em:

- **Utilizar a mesma Infraestrutura Cloud:** na fase, anterior subimos o cluster na Cloud e a aplicação rodando no Kubernetes gerenciado na nuvem de sua preferência (AWS EKS, Azure AKS, Google GKE ou Oracle OKE).
- **Pipeline de CI/CD Multi-Cloud:** automatizar o ciclo de vida da aplicação com GitHub Actions ou Azure DevOps.

- **Persistência Poliglota (NoSQL):** implementar MongoDB ou DynamoDB para lidar com dados não estruturados ou de alta escala.
- **Otimização com Cache:** implementar uma camada de cache (ex.: Redis) para reduzir latência e carga nos bancos de dados.
- **Busca Avançada:** integrar Elasticsearch (ou OpenSearch) para buscas rápidas e tolerantes a erros.

Detalhamento técnico e funcionalidades obrigatórias

Nesta fase, a exigência é a entrega de uma solução totalmente **Cloud-Native** resiliente, que utilize diferentes modelos de persistência para otimizar a performance e a experiência do usuário.

1. Infraestrutura Gerenciada e Portabilidade (mesma estrutura da fase anterior)

- Provedor de Nuvem: escolha livre entre AWS, Azure, Google Cloud (GCP) ou Oracle Cloud (OCI).
- Orquestração: provisionamento de um cluster Kubernetes Gerenciado (EKS, AKS, GKE ou OKE). Não será aceito o uso de instâncias isoladas (IaaS) para rodar containers manualmente.
- Registry: utilização de um serviço de registro de imagens privado nativo da Cloud escolhida (ex.: Amazon ECR, Azure ACR).
- Exposição: a aplicação deve ser acessível externamente via Load Balancer ou Ingress Controller configurado no cluster.

2. Persistência Poliglota e Alta Performance

- A arquitetura deve evoluir para suportar diferentes tipos de dados e cargas de trabalho, garantindo baixa latência.
- **NoSQL (Obrigatório):** implementar MongoDB ou DynamoDB.
  - Cenário de uso: deve ser utilizado para dados flexíveis ou de alta volumetria (ex.: catálogo expandido, logs de eventos, perfis de usuário ou sistema de avaliações).

- Implementação: utilizar os drivers oficiais para .NET (MongoDB.Driver ou AWSSDK.DynamoDBv2).

- **Cache de Persistência (Obrigatório):** implementar uma camada de cache distribuído (ex.: Redis).
  - Cenário de uso: persistir temporariamente dados de sessões, resultados de consultas onerosas ou configurações globais para reduzir o round-trip ao banco de dados principal.
  - Implementação: utilizar a biblioteca StackExchange.Redis ou abstrações do ASP.NET Core (IDistributedCache).

3. Busca Avançada com Elasticsearch

- O motor de busca deve ser independente da consulta tradicional do banco de dados relacional.
- Serviço: utilizar Amazon OpenSearch, Elastic Cloud ou uma instância gerenciada do Elasticsearch.
- Sincronização: implementar a lógica de indexação no CatalogAPI. Sempre que um jogo for inserido ou editado no banco principal, o índice no Elasticsearch deve ser atualizado.
- Funcionalidades: o endpoint de busca (/search) deve obrigatoriamente suportar Fuzzy Search (tolerância a erros de digitação) e ordenação por relevância.

4. Automação de CI/CD e GitOps

- Todo o ciclo de vida do software deve ser automatizado, garantindo que o código em main reflita o que está em produção.
- Ferramentas: GitHub Actions ou Azure DevOps.
- Escopo: a pipeline deve cobrir, no mínimo, os serviços UsersAPI e CatalogAPI.
- **Workflow**:
  - Build & Test: compilação do código e execução de testes unitários automatizados.

- Containerização: geração da imagem Docker com versionamento (Tags).
- Security Scan (Desejável): verificação de vulnerabilidades na imagem.
- Push: upload para o Registry da Cloud.
- Deploy: atualização do ambiente de produção (Kubernetes) sem downtime (Rolling Update).

5. Segurança e Gestão de Segredos

- Zero Hardcoded Credentials: nenhuma string de conexão, chave de API ou senha de banco de dados deve estar no código-fonte ou nos arquivos YAML.
- Injeção de Secrets: utilizar os Secrets do Kubernetes ou serviços gerenciados (AWS Secrets Manager/Azure Key Vault) integrados ao cluster para injetar as credenciais em tempo de execução.

Entregáveis da Fase 4

1. **Vídeo "Pitch Final" (até 25 min):**

- Demonstrar a infraestrutura rodando na nuvem escolhida (mostre o painel da AWS/Azure e os pods no Kubernetes).
- Realizar um Live Deploy: altere o código, faça push e mostre a pipeline atualizando a aplicação em produção automaticamente.
- Demonstrar a Busca Avançada e o impacto positivo do Cache na resposta da API.
- Explicar como o NoSQL foi integrado à arquitetura.

2. **Repositórios de Código:**

- Código atualizado com os novos drivers de NoSQL e Cache.
- Arquivos YAML de infraestrutura (Terraform, CloudFormation ou Manifestos K8s).
- Definições de Pipeline (.yml).

3. **Relatório de Entrega:**

- Nomes do grupo e usernames no Discord.
- Link da documentação (README detalhado).
- Links dos repositórios e do vídeo (YouTube ou Drive).

Lembramos que, caso você tenha qualquer dúvida, é só nos chamar no Discord!


POS TECH